
```
#For adding latex first run this otherwise the codes given below won't work
!apt-get install -y texlive texlive-latex-extra texlive-fonts-recommended dvipng cm-super
```



Show hidden output

```
#Question 1 Q1. For the population described by the transition matrix given below ([0.4,3],[0.32,0.7])
# a) What is the ratio of the total population sizes (i.e. sum of both stage classes in a generation) in
# two successive generations at equilibrium?
# b) At equilibrium what is the fraction of the juveniles in any given generation? This refers to the ratio
# of juveniles to the total population size.
import numpy as np
import matplotlib.pyplot as plt

# Define the fixed 2x2 transition matrix
transition_matrix = np.array([[0.4, 3.0],
                              [0.32, 0.7]])

# Initial population (2x1 matrix) with juveniles = 100 and adults = 0
population = np.array([[100],
                       [100]])

# Lists to store juvenile and adult populations over generations
juvenile_population = []
adult_population = []
total_population = []
ratio_total = []
ratio_juvenile = []
ratio_adult = []
iteration = 0
# Number of generations (iterations)
num_generations = 100

# Loop through each generation
for generation in range(num_generations):
    # Append the current population of juveniles and adults to respective lists
    juvenile_population.append(population[0, 0])
    adult_population.append(population[1, 0])
    total_population.append(population[0, 0] + population[1, 0])
    if iteration > 0:
        ratio_total.append((population[0, 0] + population[1, 0])/total_population[iteration-1])
        ratio_juvenile.append(100*juvenile_population[iteration-1]/total_population[iteration-1])
        ratio_adult.append(100*adult_population[iteration-1]/total_population[iteration-1])
    iteration += 1
    # Print the current population
    #print(f"Generation {generation + 1}: Juveniles = {population[0, 0]}, Adults = {population[1, 0]}")
    # Multiply the population by the transition matrix
```

```

    population = np.dot(transition_matrix, population)

# Plot the results
fig, axs = plt.subplots(1, 2, figsize=(12, 5))

# First subplot: Juvenile, Adult, and Total Population over Generations
axs[0].plot(range(num_generations), juvenile_population, label='Juveniles')
axs[0].plot(range(num_generations), adult_population, label='Adults')
axs[0].plot(range(num_generations), total_population, label='Total Population')
axs[0].set_xlabel('Generation')
axs[0].set_ylabel('Population')
axs[0].set_title('Juvenile and Adult Population Over Generations')
axs[0].legend() # To show the legend
axs[0].grid(True)

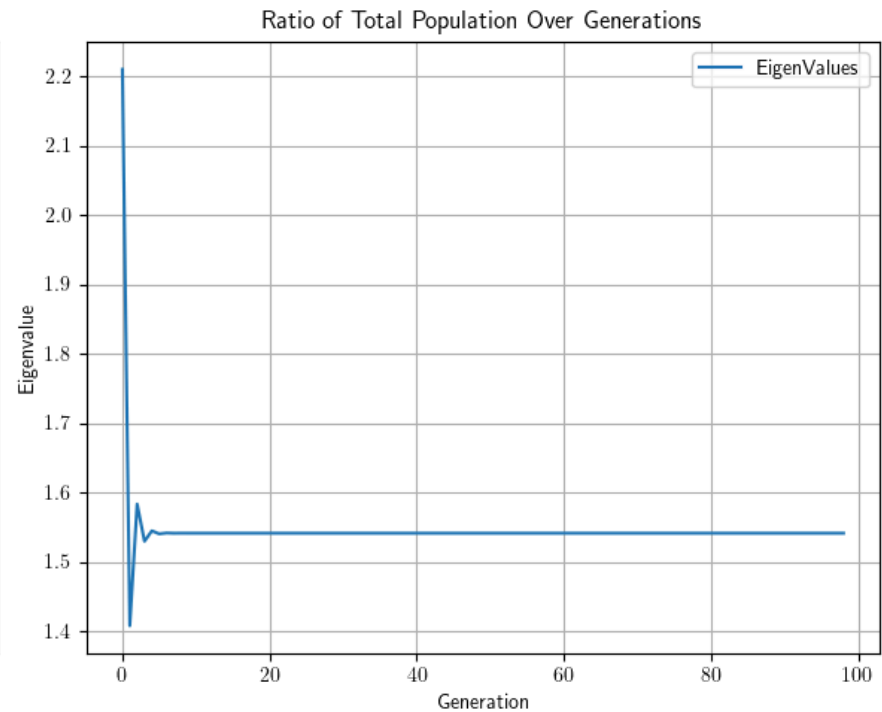
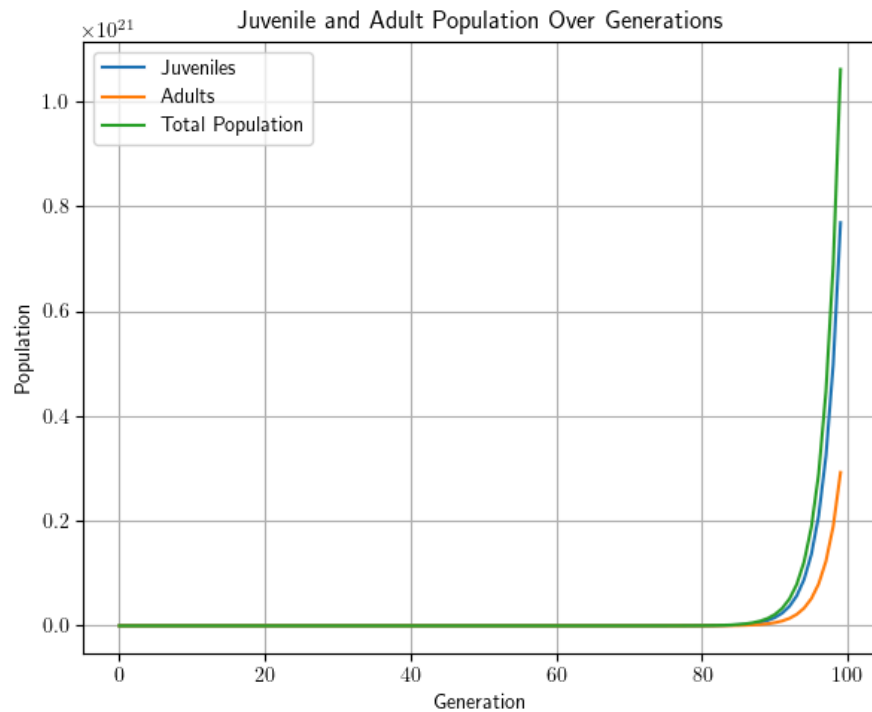
# Second subplot: Eigenvalue Ratio Over Generations
axs[1].plot(range(num_generations - 1), ratio_total, label='EigenValues')
axs[1].set_xlabel('Generation')
axs[1].set_ylabel('Eigenvalue')
axs[1].set_title('Ratio of Total Population Over Generations')
axs[1].legend() # To show the legend
axs[1].grid(True)

# Adjust layout to prevent overlap
plt.tight_layout()

# Show the plots
plt.show()

print(ratio_total)
print(f"Percentage of Juvenile: {ratio_juvenile}")
print(f"Percentage of adults: {ratio_adult}")

```



[2.21, 1.4076923076923078, 1.583060109289617, 1.5295478080773217, 1.5445758389564668, 1.540250315231796, 1.5414866812721055, 1.541132582111467, 1.5412339391776073, 1.54120492
Percentage of Juvenile: [50.0, 76.92307692307692, 71.0382513661202, 72.83396617190196, 72.32966983367561, 72.47482163651691, 72.43333284321793, 72.44521536538701, 72.44181412
Percentage of adults: [50.0, 23.076923076923077, 28.96174863387978, 27.166033828098037, 27.670330166324383, 27.525178363483082, 27.566667156782064, 27.55478463461299, 27.5581

```
# Question 2. Assume a Lotka-Volterra competition scenario with the following parameter values:
# Species 1: r1=1, K1=200, a12 = 0.8; Species 2: r2=0.5, K2=300, a21=2
# Given below are four starting points for the system. For each case, what will be the population sizes
# of Species 1 and Species 2 at equilibrium? For this your answer should consist of the filled table as
# well as the four isocline diagrams showing the trajectory of the system (not merely the vectors).
import numpy as np
import matplotlib.pyplot as plt
from pylab import rcParams
from scipy.integrate import solve_ivp

# Enable LaTeX rendering in matplotlib
plt.rcParams['text.usetex'] = True

# Set figure size
rcParams['figure.figsize'] = 10, 10

# Define the grid size for x and y axes
grid_size = 35
x = np.linspace(10, 225, grid_size) # Reduced number of points for visualization
```

```

y = np.linspace(10, 300, grid_size)

# Constants for the system
k1, k2, a12, a21, r1, r2 = (200, 300, 0.8, 2, 1, 0.5)

# Create a meshgrid for the quiver plot
xv, yv = np.meshgrid(x, y)

# Compute the vector field components using the meshgrid
x1 = r1 * xv * (k1 - xv - (a12 * yv)) / k1
y1 = r2 * yv * (k2 - yv - (a21 * xv)) / k2

# Normalize the vectors for better visibility
magnitude = np.sqrt(x1**2 + y1**2)
magnitude[magnitude == 0] = 1 # To avoid division by zero
x1_normalized = x1 / magnitude
y1_normalized = y1 / magnitude

# Create the figure and axes
fig, ax1 = plt.subplots()

# Plot the nullclines
ax1.plot(k1 - (a12 * y), y, label=r'$\frac{dN_1}{dt} = 0$')
ax1.plot(x, k2 - (a21 * x), label=r'$\frac{dN_2}{dt} = 0$')

# Add quiver plot (vector field) with normalized vectors
ax1.quiver(xv, yv, x1_normalized, y1_normalized, scale=50)

# Set axis limits and labels
ax1.set_xlim([0, 235])
ax1.set_ylim([0, 310])
ax1.set_xlabel(r'$N_1$ (K = 200)')
ax1.set_ylabel(r'$N_2$ (K = 300)')
# Define the system of equations for Lotka-Volterra
def lotka_volterra(t, z):
    N1, N2 = z
    dN1dt = r1 * N1 * (k1 - N1 - a12 * N2) / k1
    dN2dt = r2 * N2 * (k2 - N2 - a21 * N1) / k2
    return [dN1dt, dN2dt]

# Initial conditions for the trajectory
initial_conditions1 = [70, 175] # Starting population sizes for N1 and N2
initial_conditions2 = [70, 75]
initial_conditions3 = [30, 120]
initial_conditions4 = [30, 100]

# Time span for the simulation
t_span = [0, 100] # From time 0 to 100
t_eval = np.linspace(0, 100, 500) # Points at which to evaluate the solution

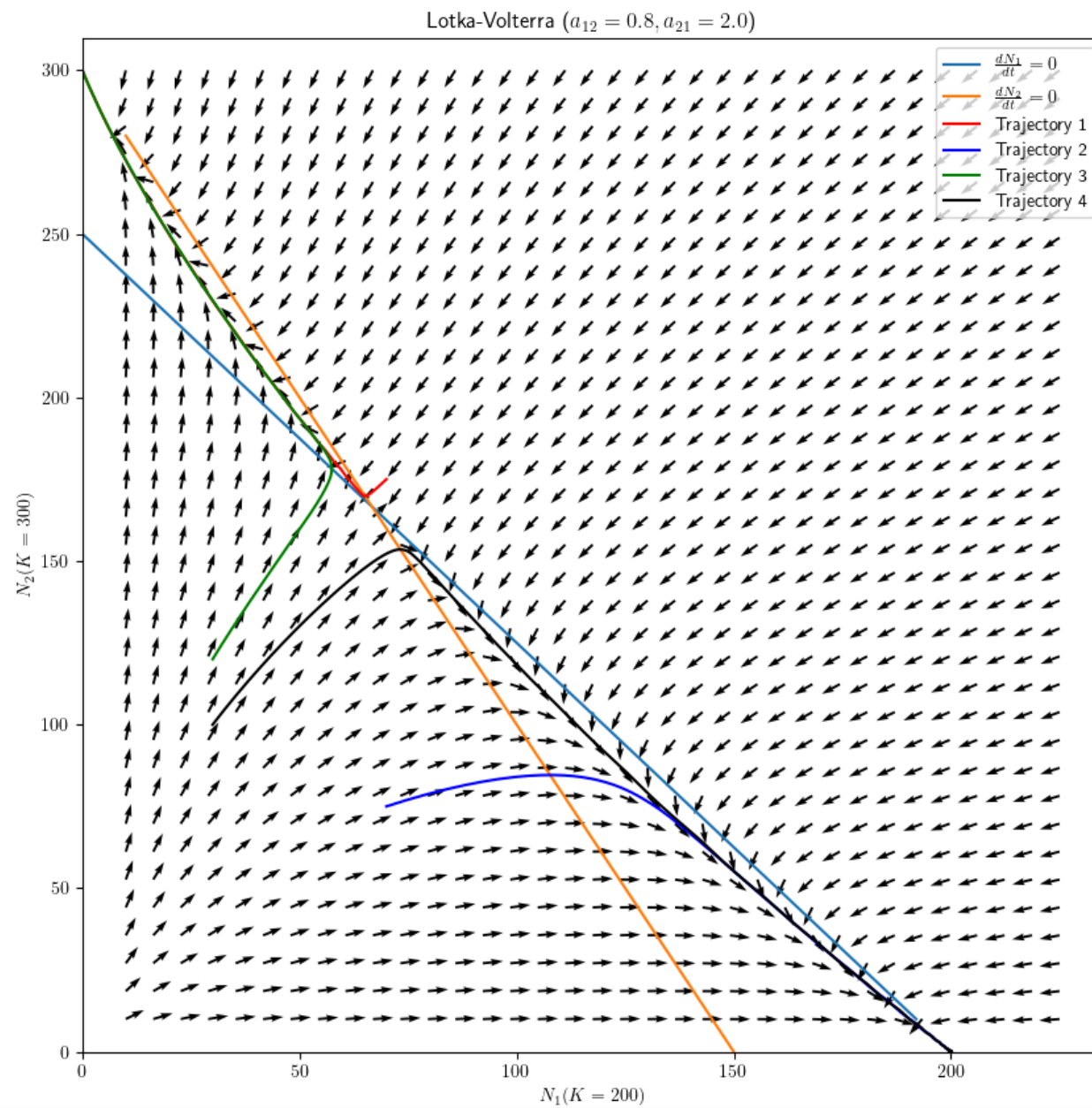
```

```
# Solve the system of equations
sol1 = solve_ivp(lotka_volterra, t_span, initial_conditions1, t_eval=t_eval)
sol2 = solve_ivp(lotka_volterra, t_span, initial_conditions2, t_eval=t_eval)
sol3 = solve_ivp(lotka_volterra, t_span, initial_conditions3, t_eval=t_eval)
sol4 = solve_ivp(lotka_volterra, t_span, initial_conditions4, t_eval=t_eval)

# Plot the trajectory
ax1.plot(sol1.y[0], sol1.y[1], color='red', label='Trajectory 1')
ax1.plot(sol2.y[0], sol2.y[1], color='blue', label='Trajectory 2')
ax1.plot(sol3.y[0], sol3.y[1], color='green', label='Trajectory 3')
ax1.plot(sol4.y[0], sol4.y[1], color='black', label='Trajectory 4')

# Add title and legend
ax1.set_title(r'Lotka-Volterra ( $a_{12} = 0.8$ ,  $a_{21} = 2.0$ )')
ax1.legend()

# Show the plot
plt.show()
```



Question 3(a). Suppose we have a hypothetical insect
population with a cohort lifetable that looks as
on the right. Over time, a population of this

```

# insect will be increasing or decreasing in
# numbers?
import numpy as np
import matplotlib.pyplot as plt

# Define the fixed 2x2 transition matrix
transition_matrix = np.zeros((12, 12))
transition_matrix[0, 3] = 0.551
transition_matrix[0, 4] = 0.412
transition_matrix[0, 5] = 0.251
transition_matrix[0, 6] = 0.99
transition_matrix[0, 7] = 0.95
transition_matrix[0, 8] = 0.71
transition_matrix[0, 9] = 0.49
transition_matrix[0, 10] = 0.135
transition_matrix[1, 0] = 239/250
transition_matrix[2, 1] = 210/239
transition_matrix[3, 2] = 156/210
transition_matrix[4, 3] = 129/156
transition_matrix[5, 4] = 109/129
transition_matrix[6, 5] = 76/109
transition_matrix[7, 6] = 43/76
transition_matrix[8, 7] = 32/43
transition_matrix[9, 8] = 19/32
transition_matrix[10, 9] = 9/19

# Initial population (2x1 matrix) with juveniles = 100 and adults = 0
population = np.array([[250],[0],[0],[0],[0],[0],[0],[0],[0],[0],[0],[0]])

# Lists to store juvenile and adult populations over generations
total_population = []
ratio_total = []
ratio_juvenile = []
ratio_adult = []
iteration = 0
# Number of generations (iterations)
num_generations = 100

# Loop through each generation
for generation in range(num_generations):

    total_population.append(population[0, 0] + population[1, 0] + population[2, 0] + population[3, 0] + population[4, 0] + population[5, 0] + population[6, 0] + population[7, 0]

    # Multiply the population by the transition matrix
    population = np.dot(transition_matrix, population)

# Plot the results

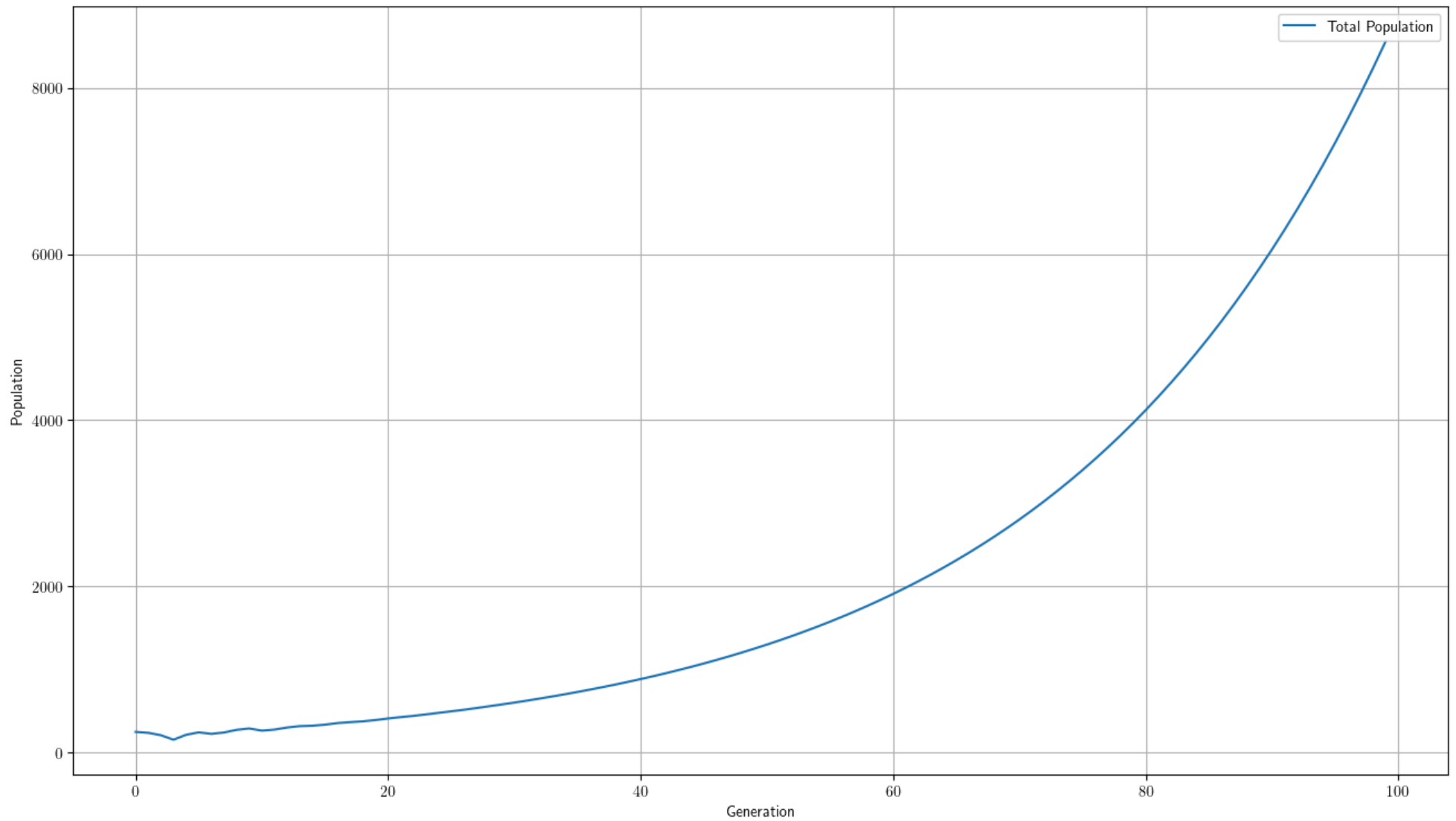
```

```
plt.figure(figsize=(16, 9))

plt.plot(range(num_generations), total_population, label='Total Population')
plt.xlabel('Generation')
plt.ylabel('Population')

plt.legend()
plt.grid(True)
plt.show()

print(f"Population Size: {total_population}")
```



Population Size: [250, 239.0, 210.0, 156.0, 214.95600000000002, 244.321936, 226.371528, 242.676068, 274.83238374399997, 290.74788727526396, 265.590858077184, 277.683965432703]

Q3b. Suppose there was a mutation in the insect
population mentioned in Part a due to which this
insect now lives two days longer with an
enhanced egg-output during late life. However,
the insect now starts reproducing one day later.
The modified life-table is given to the right. From
the perspective of long-term dynamics, is this a

```

# beneficial mutation or a harmful one?
import numpy as np
import matplotlib.pyplot as plt

# Define the fixed 2x2 transition matrix
transition_matrix = np.zeros((14, 14))
transition_matrix[0, 3] = 0
transition_matrix[0, 4] = 0.412
transition_matrix[0, 5] = 0.251
transition_matrix[0, 6] = 0.99
transition_matrix[0, 7] = 0.95
transition_matrix[0, 8] = 0.71
transition_matrix[0, 9] = 0.49
transition_matrix[0, 10] = 0.27
transition_matrix[0, 11] = 0.27
transition_matrix[0, 12] = 0.27
transition_matrix[1, 0] = 239/250
transition_matrix[2, 1] = 210/239
transition_matrix[3, 2] = 156/210
transition_matrix[4, 3] = 129/156
transition_matrix[5, 4] = 109/129
transition_matrix[6, 5] = 76/109
transition_matrix[7, 6] = 43/76
transition_matrix[8, 7] = 32/43
transition_matrix[9, 8] = 19/32
transition_matrix[10, 9] = 9/19
transition_matrix[11, 10] = 9/9
transition_matrix[12, 11] = 9/9
transition_matrix[13, 12] = 9/9

# Initial population (2x1 matrix) with juveniles = 100 and adults = 0
population = np.array([[250],[0],[0],[0],[0],[0],[0],[0],[0],[0],[0],[0],[0],[0]])

# Lists to store juvenile and adult populations over generations
total_population = []
ratio_total = []
ratio_juvenile = []
ratio_adult = []
iteration = 0
# Number of generations (iterations)
num_generations = 700

# Loop through each generation
for generation in range(num_generations):

    total_population.append(population[0, 0] + population[1, 0] + population[2, 0] + population[3, 0] + population[4, 0] + population[5, 0] + population[6, 0] + population[7, 0]

```

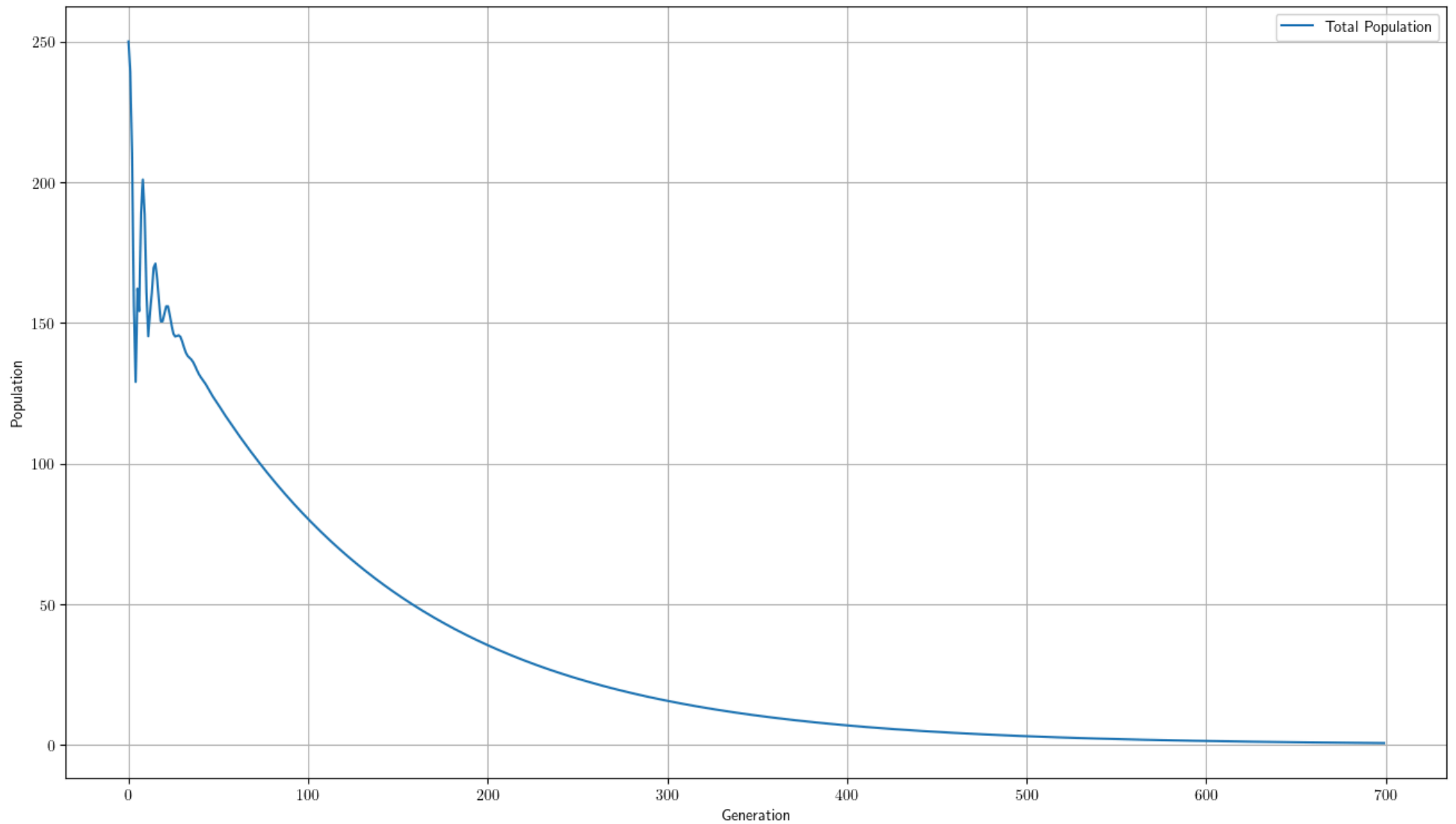
```
# Multiply the population by the transition matrix
population = np.dot(transition_matrix, population)

# Plot the results
plt.figure(figsize=(16, 9))

plt.plot(range(num_generations), total_population, label='Total Population')
plt.xlabel('Generation')
plt.ylabel('Population')

plt.legend()
plt.grid(True)
plt.show()

print(f"Population Size: {total_population}")
```



Population Size: [250, 239.0, 210.0, 156.0, 129.0, 162.148, 154.168488, 189.03952399999997, 200.92535199999998, 188.47058399999997, 160.882691616, 145.249215728896, 153.68929

Q4. The government is planning to reintroduce 10 Namibian cheetahs to Madhya Pradesh's Gandhi
Sagar Wildlife Sanctuary in January 2025. At the same time, they plan to introduce 30 Sangai deers
from Manipur into the sanctuary as prey. No further introduction of either prey or the predator is
planned. The government has decided to celebrate the introduction after a few years through a
major celebration in the park inviting many world leaders. Park authorities have determined that the
optimal viewing densities for the dignitaries would be 15 cheetahs and 18 sangais in the park. They
have also determined that the cheetah-sangai system can be modelled as a Lotka-Volterra prey-

```

# predator system
# \frac{dN}{dt} = r_1N - CNP \\
#\frac{dP}{dt} = -d_2P + gCNP
# where N and P denote the prey and the predator population respectively, and the other constants
# have their usual meaning as per the Lotka-Volterra prey-predator model. As per data from the
# scientists, the yearly rates are r1= 1.05, C = 0.1, d2=0.6 and g = 0.45. Based on this information, in
# which year will the celebratory event happen for the first time? Assume that all the assumptions of
# the Lotka-Volterra continuous prey-predator model are true for this system, and one needs to have
# both the cheetah's and the sangai's numbers to be simultaneously as per specifications.
import numpy as np
import matplotlib.pyplot as plt
from pylab import rcParams
from scipy.integrate import solve_ivp

# Enable LaTeX rendering in matplotlib
plt.rcParams['text.usetex'] = True

# Set figure size
rcParams['figure.figsize'] = 10, 10

# Define the grid size for x and y axes
grid_size = 33
x = np.linspace(1, 59, grid_size) # Reduced number of points for visualization
y = np.linspace(1, 34, grid_size)

# Constants for the system
r1, C, d2, g = (1.05, 0.1, 0.6, 0.45)

# Create a meshgrid for the quiver plot
xv, yv = np.meshgrid(x, y)

# Compute the vector field components using the meshgrid
x1 = r1 * yv - C * yv * xv
y1 = -d2 * xv + g * C * yv * xv

# Normalize the vectors for better visibility
magnitude = np.sqrt(x1**2 + y1**2)
magnitude[magnitude == 0] = 1 # To avoid division by zero
x1_normalized = x1/magnitude
y1_normalized = y1/magnitude

# Create the figure and axes
fig, ax1 = plt.subplots()

# Plot the nullclines
N_nullcline = np.linspace(5, 30, grid_size)
P_nullcline = r1 / C #- N_nullcline
P_nullcline_dP = d2 / (g * C)

# Use raw strings (r'') and remove newlines to prevent issues with LaTeX code

```

```

ax1.axvline( P_nullcline, color='blue', linestyle='--', label=r'$\frac{dN}{dt} = 0$') # Removed newline and escaped $
ax1.axhline(P_nullcline_dP, color='green', linestyle='--', label=r'$\frac{dP}{dt} = 0$') # Removed newline and escaped $

# Add quiver plot (vector field) with normalized vectors
ax1.quiver(xv, yv, x1_normalized, y1_normalized, scale=50, color='black')

# Set axis limits and labels
ax1.set_xlim([0, 60])
ax1.set_ylim([0, 35])
ax1.set_xlabel(r'$N$ (Sangai Deer)')
ax1.set_ylabel(r'$P$ (Cheetah)')

# Define the system of equations for Lotka-Volterra
def lotka_volterra(t, z):
    N, P = z
    dNdt = r1 * N - C * N * P
    dPdt = -d2 * P + g * C * N * P
    return [dNdt, dPdt]

# Initial conditions for the trajectory
initial_conditions = [10.0, 30.0] # Starting population sizes for N and P

# Time span for the simulation
t_span = [0,50] # From time 0 to 1000
t_eval = np.linspace(0, 50, 50000) # Points at which to evaluate the solution

# Solve the system of equations
sol = solve_ivp(lotka_volterra, t_span, initial_conditions, t_eval=t_eval)

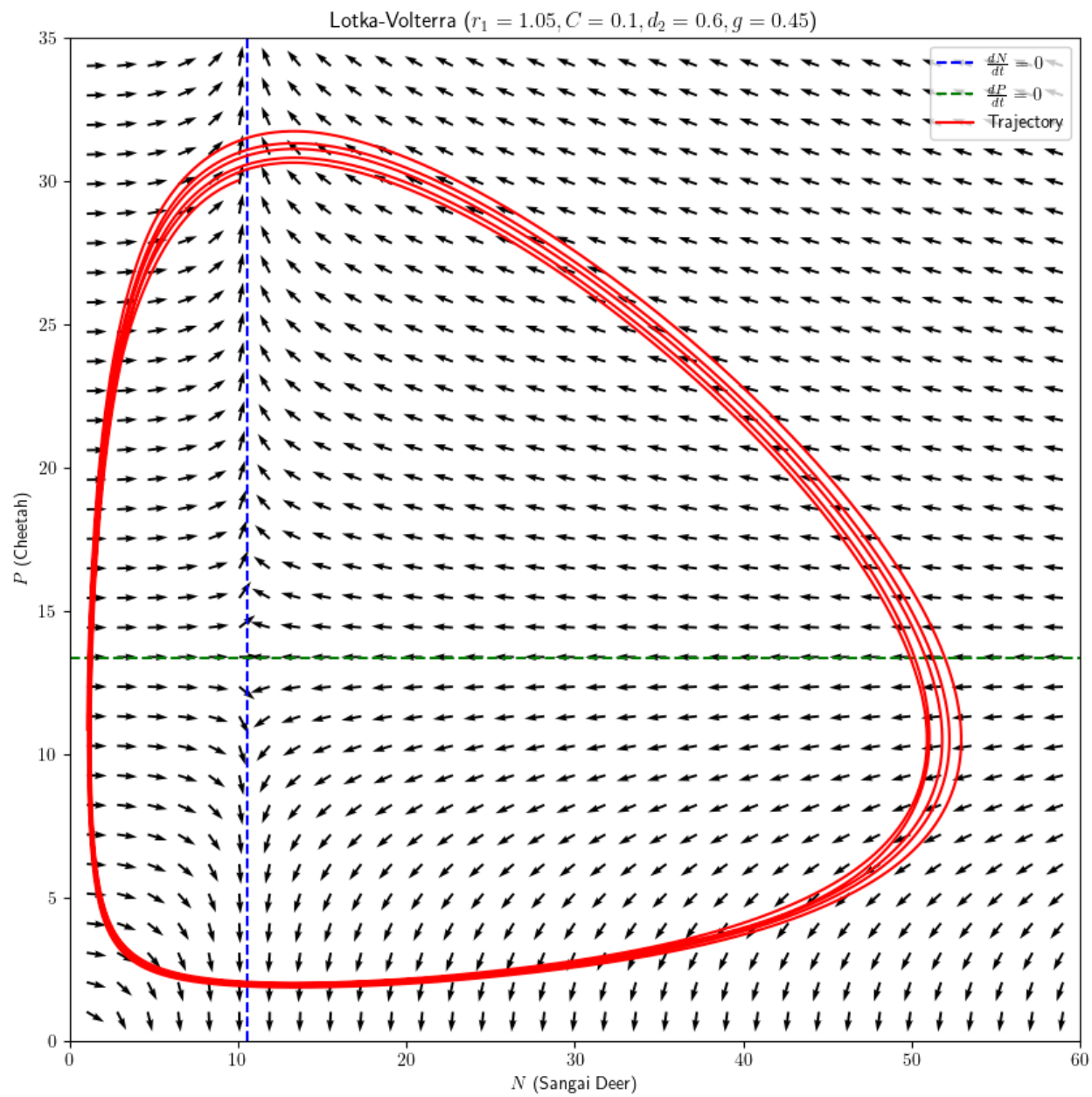
# Plot the trajectory
ax1.plot(sol.y[0], sol.y[1], color='red', label='Trajectory')

# Add title and legend
ax1.set_title(r'Lotka-Volterra ($r_1 = 1.05$, $C = 0.1$, $d_2 = 0.6$, $g = 0.45$)')
ax1.legend()

# Show the plot
plt.show()

```

14



```
import numpy as np
import matplotlib.pyplot as plt
from matplotlib import animation
```

```

from scipy.integrate import solve_ivp
from IPython.display import HTML

# Constants for the system
r1 = 1.05
C = 0.1
d2 = 0.6
g = 0.45

# Define the system of equations for Lotka-Volterra
def lotka_volterra(t, z):
    N, P = z
    dNdt = r1 * N - C * N * P
    dPdt = -d2 * P + g * C * N * P
    return [dNdt, dPdt]

# Initial conditions
initial_conditions = [30, 10] # Starting population sizes for N (Sangai Deer) and P (Cheetahs)

# Time span and evaluation points
t_span = [0, 50]
t_eval = np.linspace(0, 50, 500)

# Solve the system of equations
sol = solve_ivp(lotka_volterra, t_span, initial_conditions, t_eval=t_eval)

# Extract solutions
N_pop = sol.y[0] # Prey population (Sangai Deer)
P_pop = sol.y[1] # Predator population (Cheetah)
time = sol.t

# Set up the figure and axis
fig, ax = plt.subplots()
ax.set_xlim(0, 50)
ax.set_ylim(0, 35)
line1, = ax.plot([], [], lw=2, color='blue', label="Sangai Deer (N)") # Line for Sangai
line2, = ax.plot([], [], lw=2, color='red', label="Cheetah (P)") # Line for Cheetah
ax.set_xlabel('Time (Years)')
ax.set_ylabel('Population')
ax.set_title('Predator-Prey Population Over Time')
ax.legend()

# Adding labels for N (prey) and P (predator) populations
prey_label = ax.text(0.8, 0.9, '', transform=ax.transAxes, color='blue')
pred_label = ax.text(0.8, 0.85, '', transform=ax.transAxes, color='red')

# Function to initialize the plot
def init():
    line1.set_data([], [])
    line2.set_data([], [])

```

```

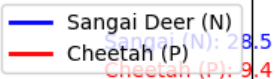
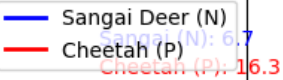
    prey_label.set_text('')
    pred_label.set_text('')
    return line1, line2, prey_label, pred_label

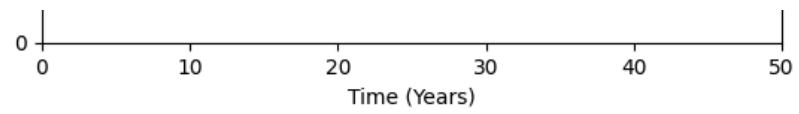
# Function to animate the plot
def animate(i):
    x = time[i]
    y1 = N_pop[i] # Prey population (Sangai Deer)
    y2 = P_pop[i] # Predator population (Cheetah)
    line1.set_data(x, y1)
    line2.set_data(x, y2)
    prey_label.set_text(f'Sangai (N): {N_pop[i]:.1f}')
    pred_label.set_text(f'Cheetah (P): {P_pop[i]:.1f}')
    return line1, line2, prey_label, pred_label

# Create the animation
ani = animation.FuncAnimation(fig, animate, init_func=init, frames=len(time), interval=20, blit=True)

# Display the animation
HTML(ani.to_jshtml())

```





```
#Thanks!!!  
a= "Thanks"  
print(a)
```

↔ Thanks